# Class 19: Online Learning and Concept Drift

Data-driven Systems Engineering

# Agenda

- Why a validated model can become obsolete.
- Offline (batch) versus online (incremental) learning.
- River: learning one observation at a time, with prequential evaluation.
- Concept drift: definition, types, and detection with ADWIN.
- Monitoring a streaming model with River and Evidently.

# Offline versus online learning

**Offline (batch)**

- Trained once on a fixed dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$.
- Straightforward benchmarking on a held-out test set.
- Adapts only through periodic retraining.

**Online (incremental)**

- Updated after each observation $(\mathbf{x}_t, y_t)$ of a stream, with constant memory.
- Operates directly on unbounded data streams.
- Adapts continuously to change.

- **Prequential evaluation** (test-then-train): each observation is first used to test the model, $\hat{y}_t = f_{t-1}(\mathbf{x}_t)$, and only then to update it, so every prediction is made on unseen data.

# Why adaptation becomes necessary

- User behaviour changes.
- Sensors degrade or are recalibrated.
- Business policies or regulations change.
- New error types appear only after deployment.

**Key point**

Adaptation does not indicate that the initial model was wrong; it acknowledges that the data-generating process evolves over time.

# River example: incremental regression

```
model = preprocessing.StandardScaler() | linear_model.LinearRegression(
    optimizer=optim.SGD(0.01)
)
mae = metrics.MAE()

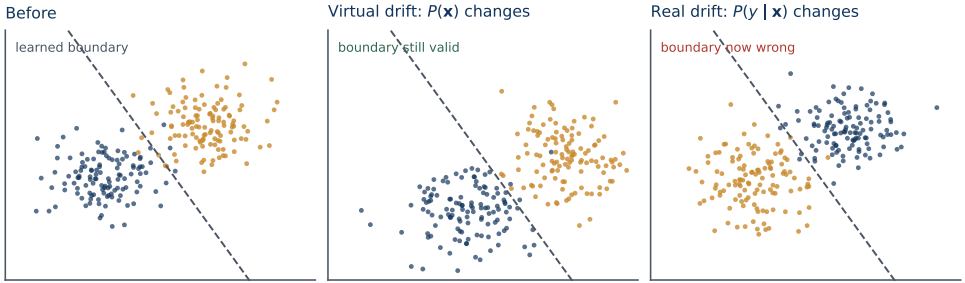
y_pred = model.predict_one(x)
model.learn_one(x, y)
mae.update(y, y_pred)
```

- From `notebooks/Class8_Model_Incremental_learning.ipynb`: the model predicts the current temperature in Paris from the wind speed (`x = {"wind": wind}`), obtained from the Open-Meteo API, which refreshes these values every 15 minutes; polling faster only repeats the same observation.
- `predict_one` precedes `learn_one` (prequential evaluation); the mean absolute error (MAE) and the parameters, by stochastic gradient descent (SGD), are updated at every observation.

# What is concept drift?

- Concept drift is a change over time of the joint distribution, $P_t(\mathbf{x}, y) \neq P_{t+\Delta}(\mathbf{x}, y)$; it is a change of the data-generating process, not noise in individual readings.
- Since $P(\mathbf{x}, y) = P(\mathbf{x})\, P(y \mid \mathbf{x})$, two cases are distinguished:
  - **virtual drift** (covariate shift): $P(\mathbf{x})$ changes, $P(y \mid \mathbf{x})$ does not;
  - **real drift**: $P(y \mid \mathbf{x})$ changes, so the learned decision function becomes incorrect.
- A model trained before a real drift may still produce confident predictions that are systematically wrong.

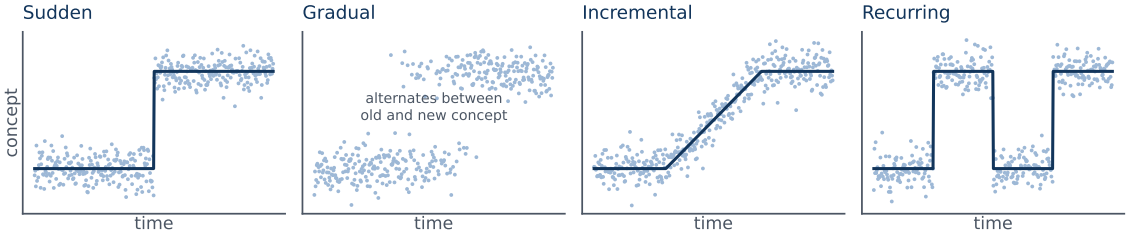
# Virtual versus real drift




- **Virtual drift:** the inputs move to a different region of $\mathcal{X}$, but the class of each point is unchanged; the learned boundary remains correct, although the model may now operate where it saw little training data.
- **Real drift:** the same inputs now belong to the other class; the boundary itself is wrong and the model must be updated.
- Monitoring the inputs alone (e.g. a data drift report) detects virtual drift; detecting real drift requires the labels, i.e. the error stream.

Illustrative synthetic data.

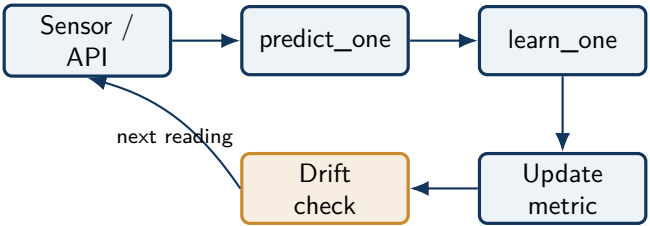
# Temporal patterns of drift




- **Sudden:** the concept is replaced at once, e.g. a sensor recalibration or a change of product.
- **Gradual:** samples from the old and the new concept alternate, with the new one becoming more frequent.
- **Incremental:** the concept moves continuously through intermediate states, e.g. wear of heating equipment.
- **Recurring:** previous concepts return, e.g. seasonal patterns; a model can reuse what it learned before.

Illustrative synthetic data; dots are observations, lines the underlying concept.

# The streaming learning loop



# River example: ADWIN drift detection

```
adwin = drift.ADWIN(delta=0.002)
...
error = abs(y - (y_pred or 0))
adwin.update(error)

if adwin.drift_detected:
    print("Resetting model due to detected drift...")
    model = preprocessing.StandardScaler() | linear_model.LinearRegression(
        optimizer=optim.SGD(0.01)
    )
    adwin = drift.ADWIN(delta=0.002)
```

- From `notebooks/Class8_Model_Incremental_learning.ipynb`: the adaptive windowing detector (ADWIN) monitors the absolute errors and signals a significant change; the notebook then resets both the model and the detector, a strategy examined experimentally below.

# How ADWIN works

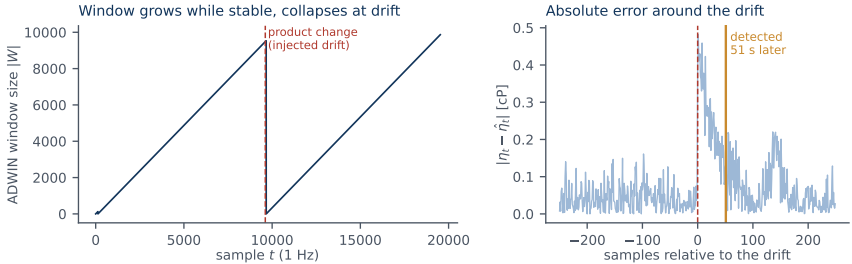
- ADWIN (Bifet and Gavaldà, 2007) maintains a window $W$ of the most recent values (here, absolute errors) and examines every split of $W$ into an older part $W_0$ and a recent part $W_1$, of sizes $n_0$ and $n_1$.
- A change is detected, and $W_0$ is dropped, when the two means differ by more than a threshold derived from the Hoeffding bound:

$$|\hat{\mu}_{W_0} - \hat{\mu}_{W_1}| \geq \varepsilon_{\text{cut}}, \qquad \varepsilon_{\text{cut}} = \sqrt{\frac{1}{2m} \ln \frac{4}{\delta'}}, \quad m = \frac{1}{1/n_0 + 1/n_1}, \quad \delta' = \frac{\delta}{n}.$$

- The confidence parameter $\delta$ (here $0.002$) controls the false-alarm rate: the window grows while the stream is stable and shrinks as soon as its mean changes.
- This provides a statistically grounded trigger for retraining or resetting, instead of a fixed retraining schedule.

# ADWIN on the pasteurization stream

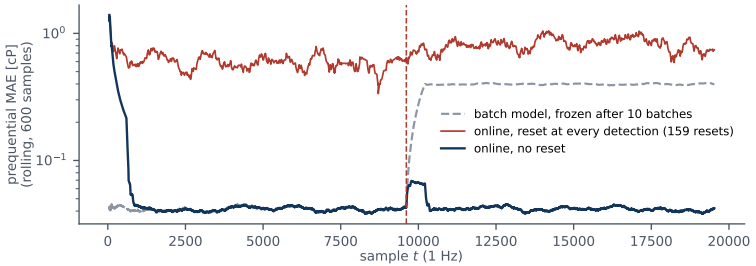
**Experiment:** an online soft sensor estimates the viscosity $\eta$ from the temperature $T$ (60 batches, 19,530 samples at 1 Hz). After batch 30, a product change raises $\eta$ by 0.4 cP at every temperature: a sudden real drift.




- The ADWIN window grows linearly while the error stream is stationary and collapses when the drift is detected, 51 samples (51 s) after it occurred.
- Apart from one alarm during the initial learning phase, no false alarm is raised in 19,530 samples ($\delta = 0.002$).

Data from `pasteurization/synth_sensors.py` with an injected drift; River 0.22, `LinearRegression` with SGD, `ADWIN(delta=0.002)`.

# Adaptation strategies compared




| Strategy | MAE before | MAE after | |
|---|---|---|---|
| Batch model, frozen after 10 batches | 0.041 | 0.398 | does not adapt |
| Online, no reset | 0.052 | 0.043 | recovers in about 200 samples (3.5 min) |
| Online, reset at every detection | 0.599 | 0.820 | 159 resets: reset loop |

- A reset replaces the model by an untrained one, whose large and decreasing error is itself a change that ADWIN detects: the model never stabilizes. Safer responses are to warm-start from the current model, retrain on the recent window $W_1$, or only raise an alert.

# Monitoring beyond a single detector

- `notebooks/Class8_Model_Incremental_learning.ipynb` also produces Evidently reports (`DataDriftPreset`, `DataQualityPreset`) alongside River.
- ADWIN on the error stream is a fast, targeted signal that detects degradation of performance; a drift report tests the distribution of every input column and can reveal virtual drift before performance degrades.
- Combining a lightweight online detector with periodic, more comprehensive reports is a common production pattern.

# Batch retraining versus online learning

| | |
|---|---|
| **Latency of adaptation** | Online learning adapts immediately; batch retraining waits for the next cycle. |
| **Auditability** | Batch models are fixed snapshots that can be reviewed and versioned; the state of an online model changes continuously. |
| **Stability** | Online models may overreact to noise if the detector or the learning rate is too sensitive. |
| **Infrastructure** | Online learning avoids large periodic training jobs but requires an always-on service. |

# Drift in the two running cases

- **Fraud:** seasonal spending patterns and new merchant categories (virtual drift); fraudsters adopting new tactics (real drift).
- **Pasteurization:** sensor recalibration (sudden), raw-material variability, or wear of heating and cooling equipment (gradual).
- In both cases, drift detection turns "the model has become worse" from an unexplained observation into a specific, timestamped event.

# Repository connections

## Repo activity

- `notebooks/Class8_Model_Incremental_learning.ipynb`: River regression, ADWIN, and Evidently reports.
- `streamlit/pages/App_Stream.py`: the same River and ADWIN pattern as a live dashboard, plotting detected change points on the MAE curve.

# Papers and materials

## Papers and materials


- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), 44:1–44:37.
- Bifet, A., Gavaldà, R. (2007). Learning from time-changing data with adaptive windowing. *Proceedings of the SIAM International Conference on Data Mining*, 443–448.
- Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., Zhang, G. (2019). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12), 2346–2363.
- Montiel, J., et al. (2021). River: machine learning for streaming data in Python. *Journal of Machine Learning Research*, 22(110), 1–8.
- Gama, J., Sebastião, R., Rodrigues, P. P. (2013). On evaluating stream learning algorithms. *Machine Learning*, 90(3), 317–346.
- Documentation: River; Evidently.

# Take-away message

The data-generating process changes over time:
evaluate models prequentially, detect drift
with a statistical test, and define in advance
how the model is updated when drift is detected.

**Thank you for your attention.**

Next: Class 20 — practical lab: online learning, concept drift, and streaming sensors.